Algorithmique

Résumé du cours BA4 – EPFL
Cours de Ola Svensson

Liam Vuilleumier

Printemps 2026

Contents

1 Lecture 1 : Introduction aux Algorithmes **3**
1.1 Qu'est-ce qu'un algorithme ? . . . 3
1.2 Exemple introductif : la somme arithmétique . . . 3
1.3 Pourquoi l'efficacité compte ? . . . 3

2 Lecture 2 : Tri et Analyse de Complexité **3**
2.1 Le problème du tri . . . 3
2.2 Tri par insertion (*Insertion Sort*) . . . 4
2.3 Preuve de correction : invariant de boucle . . . 4
2.4 Analyse de complexité (modèle RAM) . . . 4

3 Lecture 3 : Diviser pour Régner & Tri Fusion **5**
3.1 Le paradigme Diviser pour Régner . . . 5
3.2 Tri fusion (*Merge Sort*) . . . 5
3.2.1 La procédure Merge (Fusion) . . . 5
3.3 Analyse du tri fusion . . . 5

4 Lecture 4 : Analyse des Récurrences & Sous-tableau Maximum **6**
4.1 Analyse des algorithmes Diviser-pour-Régner . . . 6
4.2 Résoudre les récurrences . . . 6
4.2.1 Méthode de substitution . . . 6
4.2.2 Arbres de récursion . . . 7
4.2.3 Master Theorem (Théorème Maître) . . . 7
4.3 Problème du sous-tableau maximum . . . 7
4.3.1 Approche diviser-pour-régner . . . 7

5 Lecture 5 : Multiplication de Matrices & Tas **8**
5.1 Multiplication de matrices . . . 8
5.2 Tas (*Heaps*) et Heapsort . . . 8
5.2.1 Structure d'un tas binaire . . . 8
5.2.2 Opérations clés . . . 9

6 Lecture 6 : Heapsort & Files de Priorité **9**
6.1 L'algorithme Heapsort . . . 9
6.2 Files de priorité (*Priority Queues*) . . . 9

7 Lecture 7 : Structures de Données Élémentaires **10**

7.1 Ensembles dynamiques . . . 10
7.2 Piles (*Stacks*) – LIFO . . . 10
7.3 Files (*Queues*) – FIFO . . . 10
7.4 Listes chaînées (*Linked Lists*) . . . 11
7.4.1 Exemple : inverser une liste en $\Theta(n)$ temps, $\Theta(1)$ espace . . . 11
7.5 Conclusion sur les structures élémentaires . . . 11

8 Lecture 8 : Arbres Binaires de Recherche & Programmation Dynamique 12
8.1 Motivation : le jeu du devin . . . 12
8.2 Définition et propriété fondamentale . . . 12
8.3 Requêtes sur un ABR . . . 13
8.3.1 Recherche . . . 13
8.3.2 Minimum, Maximum, Successeur . . . 13
8.4 Parcours d'un ABR . . . 13
8.5 Modifications d'un ABR . . . 13
8.5.1 Insertion . . . 13
8.5.2 Suppression . . . 14
8.6 Bilan des opérations sur un ABR . . . 14
8.7 Comparaison des structures de données . . . 14
Programmation Dynamique . . . 14
8.8 Paradigme et idée centrale . . . 14
8.9 Premier exemple : nombres de Fibonacci . . . 15
8.9.1 Récursion naïve : arbre de calcul exponentiel . . . 15
8.9.2 Bottom-up DP : $\Theta(n)$ . . . 15
8.10 Coupe de tiges (*Rod Cutting*) . . . 16
8.10.1 Sous-structure optimale et récurrence . . . 16
8.10.2 Reconstruction de la solution . . . 16
8.11 Problème du rendu de monnaie . . . 16
8.12 Recette générale pour la PD . . . 17

9 Lecture 9 : DP – Reconstruction et Chaînes de Matrices 17
9.1 Rappel : éléments clés d'un algorithme DP . . . 17
9.2 Coupe de tiges – Reconstruction de la solution optimale . . . 17
9.2.1 Algorithme étendu avec table des décisions . . . 17
9.2.2 Exemple : tables r et s pour $n = 8$ . . . 18
9.2.3 Extraction de la solution . . . 18
9.3 Résumé : coupe de tiges . . . 19
9.4 Quand peut-on utiliser la programmation dynamique ? . . . 19
Multiplication de chaînes de matrices . . . 19
9.5 Coût de la multiplication de deux matrices . . . 19
9.6 Définition du problème . . . 20
9.7 Sous-structure optimale . . . 20
9.8 Formule récursive . . . 20
9.9 Algorithme bottom-up . . . 21
9.10 Exemple complet . . . 21
9.11 Reconstruction de la parenthésisation optimale . . . 22
9.12 Résumé : multiplication de chaînes de matrices . . . 23
9.13 Recette générale pour la DP (consolidation) . . . 23

1 Lecture 1 : Introduction aux Algorithmes

1.1 Qu'est-ce qu'un algorithme ?

Un **algorithme** est une procédure de calcul bien définie qui prend une ou plusieurs valeurs en **entrée** (*input*) et produit une ou plusieurs valeurs en **sortie** (*output*) : c'est une séquence d'étapes transformant l'entrée en sortie.

On distingue l'algorithme de la **structure de données**, qui est une méthode pour stocker et organiser l'information afin de faciliter son accès et sa modification.

1.2 Exemple introductif : la somme arithmétique

Problème : calculer $S = \sum_{i=1}^{n} i$.

Approche naïve

Boucle : additionner $1 + 2 + \cdots + n$

n additions $\Rightarrow$ $\Theta(n)$

vs.

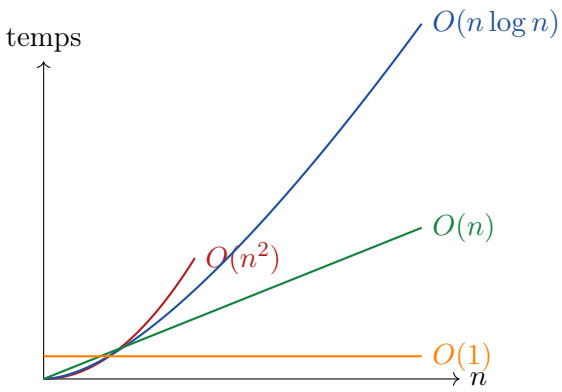
Approche de Gauss

Formule fermée : $\dfrac{n(n+1)}{2}$

3 opérations $\Rightarrow$ $\Theta(1)$

1.3 Pourquoi l'efficacité compte ?

Un ordinateur *lent* exécutant un bon algorithme finira toujours par battre un ordinateur *rapide* exécutant un mauvais algorithme, dès que la taille n de l'entrée devient suffisamment grande.



Conclusion

La **performance asymptotique** (comportement quand $n \to \infty$) est plus déterminante que les constantes multiplicatives liées au matériel.

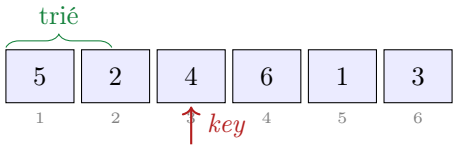
2 Lecture 2 : Tri et Analyse de Complexité

2.1 Le problème du tri

Entrée : une séquence de n nombres $\langle a_1, a_2, \ldots, a_n \rangle$.
Sortie : une permutation $\langle a'_1, a'_2, \ldots, a'_n \rangle$ telle que $a'_1 \leq a'_2 \leq \cdots \leq a'_n$.

2.2 Tri par insertion (*Insertion Sort*)

L'idée est celle du tri d'une main de cartes : on parcourt les éléments un par un et on insère chaque nouvel élément à sa place correcte parmi ceux déjà triés.



Algorithm 1 Insertion-Sort(A)

```
for j = 2 to A.length do
    key ← A[j]                          // élément à insérer
    i ← j − 1
    while i > 0 and A[i] > key do
        A[i + 1] ← A[i]                 // décaler vers la droite
        i ← i − 1
    end while
    A[i + 1] ← key                      // insérer à la bonne place
end for
```

2.3 Preuve de correction : invariant de boucle

Pour prouver qu'un algorithme itératif est correct, on utilise un **invariant de boucle**, une propriété vérifiée à trois moments clés :

1. **Initialisation** : vraie avant la première itération.
2. **Conservation** : si vraie avant une itération, reste vraie avant la suivante.
3. **Terminaison** : à la fin de la boucle, l'invariant prouve la correction.

Invariant pour le tri par insertion : au début de chaque itération j, le sous-tableau $A[1 \dots j-1]$ contient les éléments initialement en $A[1 \dots j-1]$, *mais triés*.

2.4 Analyse de complexité (modèle RAM)

On utilise le modèle **Random-Access Machine (RAM)** : chaque instruction simple (arithmétique, accès mémoire, test) prend un temps constant.

Cas	Condition	Complexité
Meilleur cas	Tableau déjà trié (while ne s'exécute jamais)	$\Theta(n)$
Pire cas	Tableau trié en ordre inverse	$\Theta(n^2)$

Calcul du pire cas :

$$T(n) = \sum_{j=2}^{n} (j-1) = \frac{n(n-1)}{2} = \Theta(n^2)$$

Pourquoi se concentrer sur le pire cas ?

Il fournit une **garantie supérieure** sur le temps d'exécution, valable pour toute entrée possible. C'est la référence standard en analyse d'algorithmes.

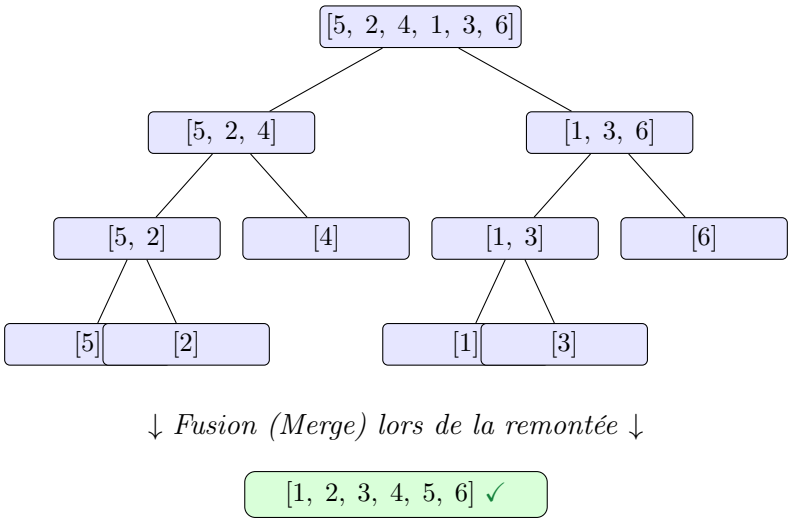
3 Lecture 3 : Diviser pour Régner & Tri Fusion

3.1 Le paradigme Diviser pour Régner

Cette approche récursive décompose tout problème en trois étapes :

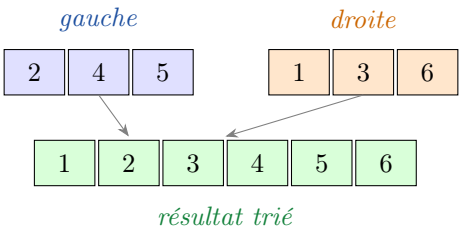
1. **Diviser** : découper le problème en sous-problèmes plus petits de même nature.
2. **Régner** : résoudre récursivement chaque sous-problème (cas de base : résolution directe si le problème est assez petit).
3. **Combiner** : fusionner les solutions partielles en une solution globale.

3.2 Tri fusion (*Merge Sort*)



3.2.1 La procédure Merge (Fusion)

L'étape clé : fusionner deux sous-tableaux déjà triés. On compare les têtes des deux listes et on place le plus petit élément en sortie. Cette opération prend un temps **linéaire** $\Theta(n)$.



3.3 Analyse du tri fusion

Soit $T(n)$ le temps pour trier n éléments :

$$T(n) = \begin{cases} \Theta(1) & \text{si } n = 1, \\ 2\,T(n/2) + \Theta(n) & \text{si } n > 1. \end{cases}$$

Arbre de récursion : à chaque niveau k, on a 2^k sous-problèmes de taille $n/2^k$, chacun contribuant $\Theta(n/2^k)$. Le coût total par niveau est $\Theta(n)$, et il y a $\log_2 n$ niveaux, donc :

niveau 0 : taille n		cn	
niveau 1 : taille $n/2$	$\times 2$	cn	
niveau 2 : taille $n/4$	$\times 4$	cn	$\log n$ niveaux
	$\vdots$	$\vdots$	
niveau $\log n$: taille 1	$\times n$	cn	

$$\boxed{T(n) = \Theta(n \log n)}$$

Comparaison Insertion Sort vs. Merge Sort

Algorithme	**Pire cas**	**En place ?**
Tri par insertion	$\Theta(n^2)$	✓
Tri fusion	$\Theta(n \log n)$	×

4 Lecture 4 : Analyse des Récurrences & Sous-tableau Maximum

4.1 Analyse des algorithmes Diviser-pour-Régner

Le temps d'exécution $T(n)$ d'un algorithme diviser-pour-régner est décrit par une **récurrence** de la forme :

$$T(n) = \begin{cases} \Theta(1) & \text{si } n \leq c, \\ a\,T(n/b) + D(n) + C(n) & \text{sinon,} \end{cases}$$

où $D(n)$ est le coût de la division et $C(n)$ celui de la combinaison. Pour le tri fusion, cela donne $T(n) = 2T(n/2) + \Theta(n)$, soit $T(n) = \Theta(n \log n)$.

4.2 Résoudre les récurrences

Il existe trois méthodes principales.

4.2.1 Méthode de substitution

On **devine** la forme de la solution, puis on utilise l'induction mathématique pour prouver les constantes.

Exemple : $T(n) = T(n/4 + 1) + T(3n/4 - 1) + \Theta(1)$. *Montrons que* $T(n) = O(n)$, *i.e.* $T(n) \leq cn - d$ *pour des constantes* $c, d > 0$.

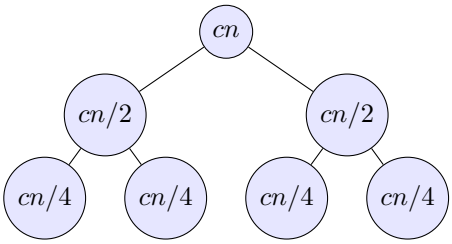
Proof. Par hypothèse d'induction, les appels récursifs vérifient la borne. Avec $\Theta(1) \leq k$:

$$\begin{aligned} T(n) &\leq c\left(\tfrac{n}{4} + 1\right) - d + c\left(\tfrac{3n}{4} - 1\right) - d + k \\ &= cn - 2d + k. \end{aligned}$$

Il suffit de choisir $d \geq k$ pour avoir $T(n) \leq cn - d$. □ □

4.2.2 Arbres de récursion

On **visualise** le coût à chaque niveau de sous-problèmes. Utile pour générer une conjecture à vérifier ensuite par substitution.



Niveau 0 : cn
Niveau 1 : $cn/2 + cn/2 = cn$
Niveau 2 : $4 \times cn/4 = cn$
$\vdots$
Total : $cn \times \log_2 n = \Theta(n \log n)$

4.2.3 Master Theorem (Théorème Maître)

Recette pour les récurrences de la forme $T(n) = aT(n/b) + f(n)$ (avec $a \geq 1$, $b > 1$). On compare $f(n)$ et $n^{\log_b a}$:

Cas	Condition sur $f(n)$	Résultat	Intuition
1	$f(n) = O\big(n^{\log_b a - \varepsilon}\big)$, $\varepsilon > 0$	$T(n) = \Theta\big(n^{\log_b a}\big)$	Les feuilles dominent
2	$f(n) = \Theta\big(n^{\log_b a}\big)$	$T(n) = \Theta\big(n^{\log_b a} \log n\big)$	Coût équiparti
3	$f(n) = \Omega\big(n^{\log_b a + \varepsilon}\big)$, $\varepsilon > 0$	$T(n) = \Theta(f(n))$	La racine domine

Exemples d'application :

- Merge Sort : $T(n) = 2T(n/2) + \Theta(n)$. Ici $a = 2$, $b = 2$, $n^{\log_2 2} = n$. Cas 2 $\Rightarrow T(n) = \Theta(n \log n)$.
- Recherche binaire : $T(n) = T(n/2) + \Theta(1)$. Ici $n^{\log_2 1} = 1$. Cas 2 $\Rightarrow T(n) = \Theta(\log n)$.
- Strassen : $T(n) = 7T(n/2) + \Theta(n^2)$. $n^{\log_2 7} \approx n^{2.81} \succ n^2$. Cas 1 $\Rightarrow T(n) = \Theta(n^{\log_2 7})$.

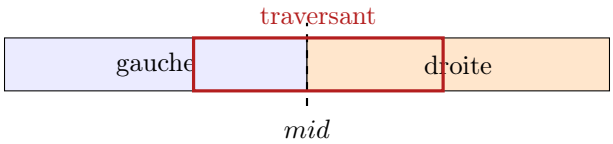
4.3 Problème du sous-tableau maximum

Entrée : un tableau $A[low \ldots high]$ de n entiers (certains négatifs).
Sortie : les indices i, j et la somme $\sum_{k=i}^{j} A[k]$ maximale.

4.3.1 Approche diviser-pour-régner

On divise A en deux moitiés $A[low \ldots mid]$ et $A[mid{+}1 \ldots high]$. Le sous-tableau maximal est soit :

1. Entièrement dans la moitié **gauche**,
2. Entièrement dans la moitié **droite**,
3. **Traversant le milieu** (une partie à gauche de mid, une à droite).



La fonction FIND-MAX-CROSSING-SUBARRAY scanne depuis *mid* vers la gauche puis vers la droite en $\Theta(n)$. La récurrence globale est :

$$T(n) = 2T(n/2) + \Theta(n) \;\Rightarrow\; \boxed{T(n) = \Theta(n \log n)}$$

5 Lecture 5 : Multiplication de Matrices & Tas

5.1 Multiplication de matrices

Problème : étant données deux matrices $n \times n$ A et B, calculer $C = A \cdot B$ où $c_{ij} = \sum_{k=1}^{n} a_{ik} b_{kj}$.

Algorithme	Récurrence	Complexité	Remarque
Naïf (3 boucles imbriquées)	—	$\Theta(n^3)$	Référence
D&C simple (8 sous-problèmes)	$T(n) = 8T(n/2) + \Theta(n^2)$	$\Theta(n^3)$	Pas d'amélioration
Strassen	$T(n) = 7T(n/2) + \Theta(n^2)$	$\Theta(n^{\log_2 7}) \approx \Theta(n^{2.81})$	Mieux !

L'idée de Strassen : utiliser des combinaisons algébriques astucieuses pour n'effectuer que **7 multiplications** de sous-matrices au lieu de 8, au prix de quelques additions supplémentaires (moins coûteuses).

Application du Master Theorem à Strassen

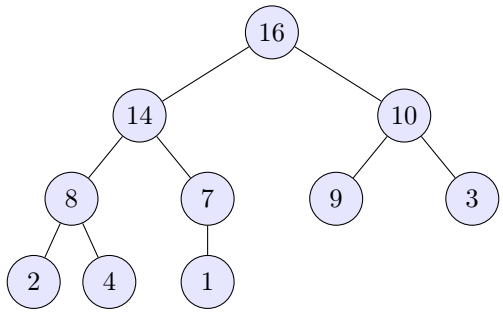
$a = 7$, $b = 2$, $f(n) = \Theta(n^2)$, $n^{\log_2 7} \approx n^{2.81}$.
Comme $f(n) = O(n^{\log_2 7 - \varepsilon})$ (cas 1) : $T(n) = \Theta(n^{\log_2 7}) \approx \Theta(n^{2.81})$.

5.2 Tas (*Heaps*) et Heapsort

Le Heapsort combine les avantages du tri fusion ($O(n \log n)$) et du tri par insertion (tri *en place*).

5.2.1 Structure d'un tas binaire

Un **tas (max-heap)** est un arbre binaire *presque complet* stocké dans un tableau.



$A =$	16	14	10	8	7	9	3	2	4	1
	1	2	3	4	5	6	7	8	9	10

Pour un nœud à l'indice i :

$$\text{PARENT}(i) = \lfloor i/2 \rfloor, \quad \text{LEFT}(i) = 2i, \quad \text{RIGHT}(i) = 2i + 1.$$

Propriété max-heap : $A[\text{PARENT}(i)] \geq A[i]$ pour tout $i \neq$ racine. Le maximum est toujours à la racine $A[1]$.

5.2.2 Opérations clés

Max-Heapify(A, i)**.** Restaure la propriété max-heap pour le sous-arbre enraciné en i, en supposant que les sous-arbres gauche et droit sont déjà des max-heaps. L'élément « coule » vers le bas. Complexité : $O(\log n)$ (hauteur du tas).

Build-Max-Heap(A)**.** Construit un max-heap à partir d'un tableau quelconque en appelant Max-Heapify sur tous les nœuds non-feuilles (de $\lfloor n/2 \rfloor$ jusqu'à 1).

Algorithm 2 Build-Max-Heap(A)

```
A.heap_size ← A.length
for i ← ⌊A.length/2⌋ downto 1 do
    Max-Heapify(A, i)
end for
```

Complexité de Build-Max-Heap

Bien que l'on appelle $O(n)$ fois une fonction en $O(\log n)$, l'analyse fine (la majorité des nœuds ont une faible hauteur) donne :

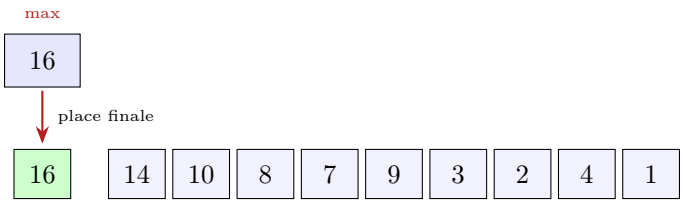
$$\boxed{T_{\text{build}} = \Theta(n)}$$

6 Lecture 6 : Heapsort & Files de Priorité

6.1 L'algorithme Heapsort

Heapsort trie un tableau *en place* en $O(n \log n)$ dans le pire cas.

1. **Construire** un max-heap depuis le tableau d'entrée : $O(n)$.
2. **Répéter** $n - 1$ **fois :**
   a. Échanger la racine (maximum) $A[1]$ avec le dernier élément $A[heap_size]$.
   b. Réduire $heap_size$ de 1 (l'élément échangé est à sa place définitive).
   c. Appeler Max-Heapify$(A, 1)$ pour restaurer la propriété : $O(\log n)$.



Complexité 6.1: Heapsort

- Build-Max-Heap : $O(n)$.
- $n - 1$ appels à Max-Heapify : $O(\log n)$ chacun.
- **Total :** $O(n \log n)$ **dans tous les cas.**

6.2 Files de priorité (*Priority Queues*)

Une **file de priorité (max)** maintient un ensemble dynamique d'éléments avec clé, et supporte :

Opération	Description	Temps
MAXIMUM(S)	Retourne l'élément de clé maximale	$O(1)$
EXTRACT-MAX(S)	Retire et retourne le maximum	$O(\log n)$
INCREASE-KEY(S, x, k)	Augmente la clé de x à k	$O(\log n)$
INSERT(S, x)	Insère x dans S	$O(\log n)$

Implémentation : le tas binaire implémente toutes ces opérations efficacement.

EXTRACT-MAX.

1. Sauvegarder $A[1]$ (le maximum).
2. Mettre $A[heap_size]$ à la racine, réduire $heap_size$.
3. Appeler MAX-HEAPIFY$(A, 1)$.

INCREASE-KEY(A, i, k). Après avoir mis à jour $A[i] \leftarrow k$, remonter dans l'arbre tant que la propriété max-heap est violée (l'élément « flotte » vers le haut).

Application : file d'attente à priorité

Utilisée dans les algorithmes de graphes (Dijkstra, Prim) pour extraire efficacement le prochain sommet à traiter.

7 Lecture 7 : Structures de Données Élémentaires

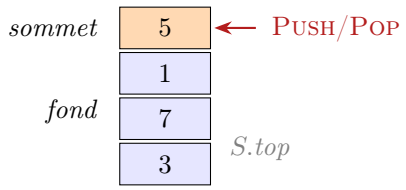
7.1 Ensembles dynamiques

Les structures de données servent à implémenter des **ensembles dynamiques**. Les opérations se regroupent en deux catégories :

- **Modification** : Insert, Delete, ...
- **Requête** : Search, Minimum, Maximum, ...

7.2 Piles (*Stacks*) – LIFO

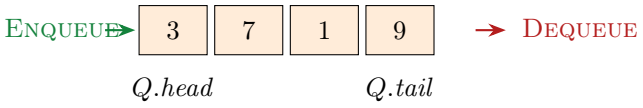
Une pile suit le principe **LIFO** (*Last-In, First-Out*) : dernier entré, premier sorti, comme une pile d'assiettes.



- PUSH(S, x) : empile x en haut. $O(1)$.
- POP(S) : retire et retourne le sommet. $O(1)$.
- STACK-EMPTY(S) : teste si $S.top = 0$. $O(1)$.

7.3 Files (*Queues*) – FIFO

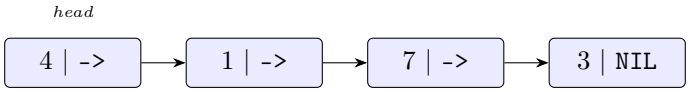
Une file suit le principe **FIFO** (*First-In, First-Out*) : premier entré, premier sorti, comme une file d'attente.



- ENQUEUE(Q, x) : ajoute x en queue. $O(1)$.
- DEQUEUE(Q) : retire et retourne l'élément de tête. $O(1)$.

7.4 Listes chaînées (*Linked Lists*)

L'ordre linéaire est déterminé par des **pointeurs** stockés dans chaque objet (et non par des indices de tableau).



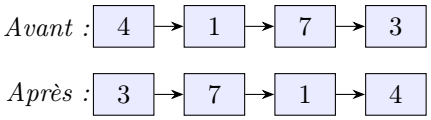
Opération	Temps	Remarque
LIST-SEARCH(L, k)	$O(n)$	Parcours séquentiel
LIST-INSERT(L, x)	$O(1)$	Insertion en tête
LIST-DELETE(L, x)	$O(1)$	Si pointeur déjà connu (liste doublement chaînée)

Sentinelles : un objet factice `L.nil` représente simultanément les extrémités de la liste, simplifiant les cas limites (pas de test `NULL`).

7.4.1 Exemple : inverser une liste en $\Theta(n)$ temps, $\Theta(1)$ espace

Algorithm 3 Reverse-List(L)

```
prev ← NIL
curr ← L.head
while curr ≠ NIL do
    next ← curr.next
    curr.next ← prev                    // inverser le lien
    prev ← curr
    curr ← next
end while
L.head ← prev
```



7.5 Conclusion sur les structures élémentaires

Forces et limites

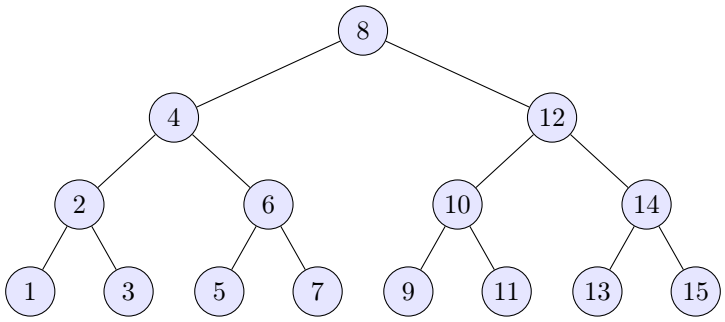
Piles, files et listes chaînées excellent pour les insertions/suppressions en $O(1)$, mais sont **mauvaises pour la recherche** ($O(n)$). Pour une recherche rapide, il faut des structures plus sophistiquées (arbres, tables de hachage).

8 Lecture 8 : Arbres Binaires de Recherche & Programmation Dynamique

8.1 Motivation : le jeu du devin

Ola pense à un entier entre 1 et 15. À chaque devinette, il répond *correct*, *plus petit* ou *plus grand*.

Stratégie optimale : toujours deviner le milieu de l'intervalle restant. Cela garantit de trouver n'importe quel nombre en au plus $\lceil \log_2 15 \rceil = 4$ questions. Cette stratégie s'encode naturellement sous la forme d'un arbre :



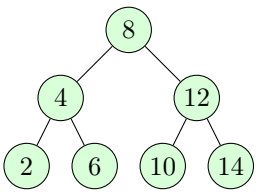
8.2 Définition et propriété fondamentale

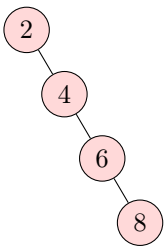
Définition 8.1: Arbre binaire de recherche (ABR)

Un **ABR** est un arbre binaire T où chaque nœud x vérifie :

- Si y est dans le *sous-arbre gauche* de x : $y.key < x.key$.
- Si y est dans le *sous-arbre droit* de x : $y.key \geq x.key$.

La **hauteur** h est le nombre d'arêtes du plus long chemin racine–feuille.





Équilibré : $h = 2$, ops en $O(\log n)$ Dégénéré : $h = n - 1$, ops en $O(n)$

Complexité 8.1: Toutes les opérations sur un ABR

Recherche, insertion, suppression, min, max, successeur, prédécesseur : toutes en $O(h)$. Pour un arbre équilibré : $h = O(\log n)$.

8.3 Requêtes sur un ABR

8.3.1 Recherche

Algorithm 4 BST-Search(T, k)

```
x ← T.root
while x ≠ NIL and k ≠ x.key do
    if k < x.key then
        x ← x.left
    else
        x ← x.right
    end if
end while
return x
```

8.3.2 Minimum, Maximum, Successeur

Théorème 8.1: Localisation de min/max

- Le **minimum** est le nœud le plus à *gauche*.
- Le **maximum** est le nœud le plus à *droite*.

Le **successeur** de x (plus petite clé $> x.key$) se trouve :

1. Si x a un sous-arbre droit : c'est le *minimum* du sous-arbre droit.
2. Sinon : remonter jusqu'au premier ancêtre y tel que x est dans le sous-arbre *gauche* de y.

8.4 Parcours d'un ABR

Parcours	Ordre	Résultat (ex.)	Utilité
Infixe (inorder)	gauche, racine, droite	$1, 2, \ldots, n$ (trié !)	Tri
Préfixe (preorder)	racine, gauche, droite	racine en premier	Copie
Postfixe (postorder)	gauche, droite, racine	racine en dernier	Suppression

Point Clé 8.1: Parcours infixe et tri

Le parcours infixe d'un ABR produit toujours les clés en **ordre croissant** (propriété directe de la définition de l'ABR). Complexité : $\Theta(n)$.

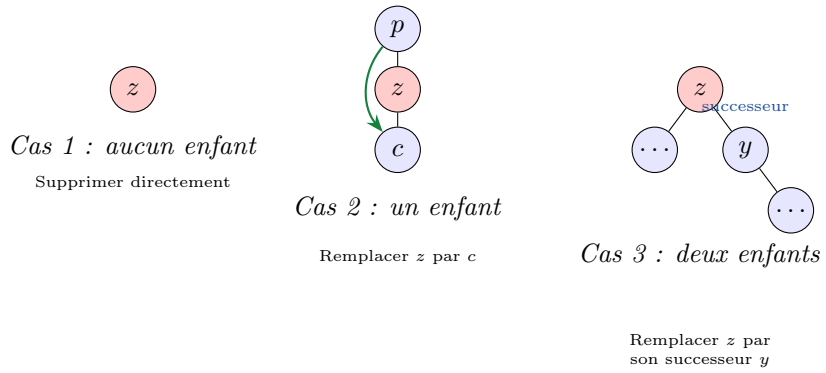
8.5 Modifications d'un ABR

8.5.1 Insertion

Chercher $z.key$ jusqu'à atteindre NIL, puis insérer z à cet endroit. Complexité : $O(h)$.

8.5.2 Suppression

Trois cas selon le nombre d'enfants du nœud z à supprimer :



8.6 Bilan des opérations sur un ABR

Opération	Temps	Idée clé
Recherche	$O(h)$	Suivre gauche/droite selon la clé
Minimum / Maximum	$O(h)$	Nœud le plus à gauche/droite
Successeur / Préd.	$O(h)$	Min du s.a. droit, ou premier ancêtre
Insertion	$O(h)$	Phase de recherche + insertion feuille
Suppression	$O(h)$	3 cas ; utilise TRANSPLANT
Parcours infixe	$\Theta(n)$	Visite chaque nœud une fois

Point Clé 8.2: Équilibrage

Dans le pire cas $h = \Theta(n)$ (arbre dégénéré). Les arbres auto-équilibrés (AVL, rouge-noir) maintiennent $h = O(\log n)$, garantissant $O(\log n)$ par opération.

8.7 Comparaison des structures de données

Structure	Capacité	Insertion	Suppression	Recherche
Pile / File (tableau)	fixe	$O(1)$	$O(1)$	$O(n)$
Liste chaînée	dynamique	$O(1)$	$O(1)$	$O(n)$
ABR équilibré	dynamique	$O(\log n)$	$O(\log n)$	$O(\log n)$

Programmation Dynamique

8.8 Paradigme et idée centrale

Définition 8.2: Programmation dynamique (PD)

La **programmation dynamique** est un paradigme algorithmique (pas une fonctionnalité de langage). Son principe :

Mémoriser les calculs déjà effectués pour éviter de les refaire.

Cela permet de transformer des algorithmes exponentiels en algorithmes polynomiaux.

Un algorithme DP repose sur deux propriétés :

1. **Sous-structure optimale** : la solution optimale d'un problème contient les solutions optimales de ses sous-problèmes.
2. **Sous-problèmes répétés** : un algorithme récursif naïf recalcule les mêmes sous-problèmes un nombre exponentiel de fois.

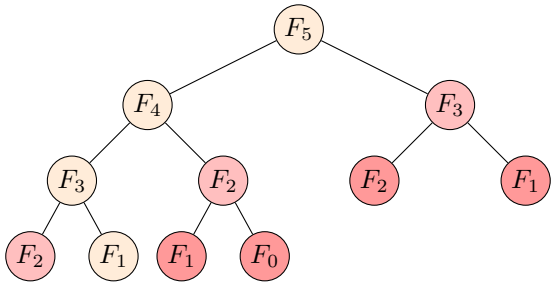
Deux stratégies d'implémentation :

- **Top-down avec mémoïsation** : récursion + table de résultats.
- **Bottom-up** : remplir la table des petits sous-problèmes vers les grands.

8.9 Premier exemple : nombres de Fibonacci

$$F_0 = 1, \quad F_1 = 1, \quad F_n = F_{n-1} + F_{n-2} \quad (n \geq 2).$$

8.9.1 Récursion naïve : arbre de calcul exponentiel



Les nœuds rouges sont recalculés plusieurs fois !

Complexité : **exponentielle** $O(2^n)$.

8.9.2 Bottom-up DP : $\Theta(n)$

Algorithm 5 Bottom-Up-Fib(n)

```
r[0] ← 1;  r[1] ← 1
for i = 2 to n do
    r[i] ← r[i − 1] + r[i − 2]
end for
return r[n]
```

1	1	2	3	5	8	13	21
$r[0]$	$r[1]$	$r[2]$	$r[3]$	$r[4]$	$r[5]$	$r[6]$	$r[7]$

8.10 Coupe de tiges (*Rod Cutting*)

Définition 8.3: Problème de la coupe de tiges

Entrée : une tige de longueur n et un tableau de prix p_i pour une tige de longueur i ($1 \leq i \leq n$).
Objectif : trouver la découpe maximisant le revenu total.

Il existe 2^{n-1} découpages possibles : la force brute est inenvisageable.

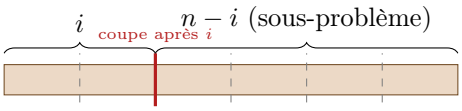
8.10.1 Sous-structure optimale et récurrence

Théorème 8.2: Structure optimale

Si la première coupe optimale est après i unités, et si $s_1, \ldots, s_k$ est un découpage optimal de la tige restante de longueur $n-i$, alors $i, s_1, \ldots, s_k$ est optimal pour la tige de longueur n.

Soit $r(n)$ le revenu optimal pour une tige de longueur n :

$$r(n) = \begin{cases} 0 & \text{si } n = 0, \\ \max\limits_{1 \leq i \leq n} \{p_i + r(n-i)\} & \text{si } n \geq 1. \end{cases}$$



Complexité 8.2: DP pour la coupe de tiges

- Top-down avec mémoïsation : $O(n^2)$ (graphe de sous-problèmes).
- Bottom-up : $O(n^2)$.

8.10.2 Reconstruction de la solution

On maintient un tableau $s[j]$ stockant la première coupe optimale pour $r(j)$. Ensuite, on traceback : couper $s[n]$, puis résoudre sur $n - s[n]$.

i	0	1	2	3	4	5	6	7	8
$r[i]$	0	1	5	8	10	13	17	18	22
$s[i]$	0	1	2	3	2	2	6	1	2

8.11 Problème du rendu de monnaie

Définition 8.4: Rendu de monnaie

Entrée : n dénominations $0 < w_1 < \cdots < w_n$ et un montant cible W.
Sortie : nombre minimum de pièces pour atteindre exactement W.

Récurrence DP :

$$c(w) = \begin{cases} 0 & \text{si } w = 0, \\ \min\limits_{\substack{1 \leq j \leq n \\ w_j \leq w}} \{1 + c(w - w_j)\} & \text{si } w \geq 1. \end{cases}$$

Exemple : $w_1 = 1, w_2 = 2, w_3 = 5$, $W = 8$. Optimal : 1 pièce de chaque $\Rightarrow$ 3 pièces au total.

Remplissage bottom-up de $w = 0$ à W : complexité $O(nW)$.

8.12 Recette générale pour la PD

Étapes de conception d'un algorithme DP

1. **Identifier les choix et la sous-structure optimale** : un choix réduit le problème à de plus petits sous-problèmes, dont les solutions optimales composent la solution globale.
2. **Écrire une récurrence** pour la valeur optimale en fonction des sous-problèmes.
3. **Éviter les recalculs** via mémoïsation (top-down) ou remplissage de table (bottom-up).
4. **(Optionnel) Reconstruire la solution** en stockant les choix dans une table auxiliaire et en remontant le tracé.

Problème	Complexité	Taille de la table
Fibonacci	$\Theta(n)$	$n + 1$ entrées
Coupe de tiges	$O(n^2)$	$n + 1$ entrées
Rendu de monnaie	$O(nW)$	$W + 1$ entrées

9 Lecture 9 : DP – Reconstruction et Chaînes de Matrices

9.1 Rappel : éléments clés d'un algorithme DP

La programmation dynamique repose sur deux propriétés fondamentales :

1. **Sous-structure optimale** : une solution optimale s'obtient en faisant un *choix*, ce qui laisse un ou plusieurs sous-problèmes à résoudre, et la solution optimale globale résout ces sous-problèmes de façon optimale.
2. **Sous-problèmes répétés** : un algorithme récursif naïf recalcule les mêmes sous-problèmes de nombreuses fois.
   - **Top-down avec mémoïsation** : résoudre récursivement en mémorisant chaque résultat dans une table.
   - **Bottom-up** : trier les sous-problèmes et résoudre d'abord les plus petits ; ainsi, quand on résout un sous-problème, les sous-problèmes dont il dépend sont déjà calculés.

9.2 Coupe de tiges – Reconstruction de la solution optimale

En Lecture 8, l'algorithme DP renvoyait uniquement le revenu optimal $r(n)$. Il est souvent nécessaire de récupérer *comment* obtenir ce revenu, c'est-à-dire les longueurs des morceaux à couper.

9.2.1 Algorithme étendu avec table des décisions

L'idée est de maintenir une table auxiliaire $s[0 \dots n]$: $s[j]$ stocke la longueur du premier morceau coupé (la *décision*) correspondant au revenu optimal $r[j]$.

Algorithm 6 Extended-Bottom-Up-Cut-Rod(p, n)

```
Créer les tableaux r[0...n] et s[0...n]
r[0] ← 0
for j = 1 to n do
    q ← −∞
    for i = 1 to j do
        if q < p[i] + r[j − i] then
            q ← p[i] + r[j − i]                    // meilleur revenu
            s[j] ← i                               // stocker le choix optimal
        end if
    end for
    r[j] ← q
end for
return r et s
```

La complexité reste $\Theta(n^2)$. La seule différence avec l'algorithme de base est le maintien de $s[j]$ qui enregistre l'indice i réalisant le maximum.

9.2.2 Exemple : tables r et s pour $n = 8$

En utilisant la table de prix p_i : 1, 5, 8, 9, 10, 17, 17, 20, 24, 30 :

i	0	1	2	3	4	5	6	7	8
$r[i]$	0	1	5	8	10	13	17	18	22
$s[i]$	0	1	2	3	2	2	6	1	2

9.2.3 Extraction de la solution

La table s stocke les choix menant à une solution optimale. On remonte la solution en traçant-back depuis n :

Algorithm 7 Print-Cut-Rod-Solution(p, n)

```
(r, s) ← EXTENDED-BOTTOM-UP-CUT-ROD(p, n)
while n > 0 do
    afficher s[n]                                  // taille du prochain morceau
    n ← n − s[n]
end while
```

Trace pour $n = 8$:

$n = 8$ **sortie :** $\mathbf{2}n = 6$ **sortie :** $\mathbf{6}n = 0$

Découpe : **2** + **6**, revenu $= 5 + 17 = 22$

9.3 Résumé : coupe de tiges

Récapitulatif DP – Coupe de tiges

- **Choix** : où effectuer la coupe la plus à gauche.
- **Sous-structure optimale** : le reste doit être découpé de façon optimale.
- **Récurrence** :

$$r(n) = \begin{cases} 0 & \text{si } n = 0, \\ \max\limits_{1 \le i \le n} \{p_i + r(n-i)\} & \text{si } n \ge 1. \end{cases}$$

- **Complexité** : $\Theta(n^2)$ (top-down ou bottom-up).
- **Reconstruction** : table auxiliaire s + trace-back en $O(n)$.

9.4 Quand peut-on utiliser la programmation dynamique ?

1. **Sous-structure optimale.**
   - Une solution optimale peut être construite en combinant des solutions optimales de sous-problèmes.
   - Cela implique que la valeur optimale peut être exprimée par une *formule récursive*.
2. **Sous-problèmes répétés.**
   - Un algorithme récursif naïf peut revisiter le même sous-problème de nombreuses fois.

Point Clé 9.1: Problème du rendu de monnaie

La DP s'applique directement à ce problème classique. Avec n dénominations $0 < w_1 < \cdots < w_n$ et un montant cible W, on minimise le nombre de pièces :

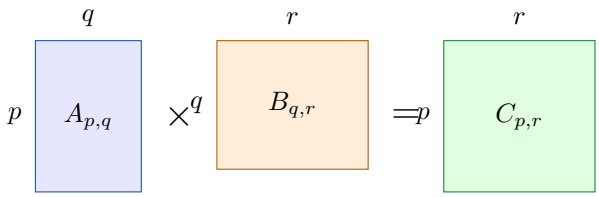
$$\min \left\{ \sum_{j=1}^{n} x_j \ : \ \sum_{j=1}^{n} w_j x_j = W, \ x_j \in \mathbb{Z}_{\ge 0} \right\}.$$

Exemple : $w_1 = 1$, $w_2 = 2$, $w_3 = 5$, $W = 8$. Optimal : $x_1 = x_2 = x_3 = 1 \Rightarrow 3$ pièces.

Multiplication de chaînes de matrices

9.5 Coût de la multiplication de deux matrices

Multiplier une matrice $A_{p\times q}$ par une matrice $B_{q\times r}$ produit une matrice $C_{p\times r}$.



Coût : $p \cdot q \cdot r$ multiplications scalaires

Chaque entrée de C nécessite q multiplications scalaires. La matrice C contient $p \cdot r$ entrées, donc le coût total est $\mathbf{p \cdot q \cdot r}$ **multiplications scalaires**.

Pourquoi l'ordre d'association importe-t-il ?

Pour A (20×1), B (1×10), C (10×100) :

- $(A \times (B \times C))$: $1 \cdot 10 \cdot 100 + 20 \cdot 1 \cdot 100 = 3\,000$
- $((A \times B) \times C)$: $20 \cdot 1 \cdot 10 + 20 \cdot 10 \cdot 100 = 20\,200$

La parenthésisation peut **modifier radicalement** le nombre d'opérations.

9.6 Définition du problème

Définition 9.1: Multiplication de chaînes de matrices

Entrée : une chaîne $\langle A_1, A_2, \ldots, A_n \rangle$ de n matrices, où la matrice A_i a la dimension $p_{i-1} \times p_i$ pour $i = 1, \ldots, n$.
Sortie : une parenthésisation complète du produit $A_1 A_2 \cdots A_n$ minimisant le nombre de multiplications scalaires.

Remarques :

- On ne demande *pas* de calculer le produit, seulement de trouver la meilleure parenthésisation.
- La parenthésisation peut affecter considérablement le nombre de multiplications.

Exemple : chaîne $A_1 A_2 A_3$ avec dimensions 50×5, 5×100, 100×10 :

Parenthésisation	Coût	Calcul
$(A_1 A_2) A_3$	75 000	$50 \cdot 5 \cdot 100 + 50 \cdot 100 \cdot 10$
$A_1 (A_2 A_3)$	7 500	$5 \cdot 100 \cdot 10 + 50 \cdot 5 \cdot 10$

9.7 Sous-structure optimale

Théorème 9.1: Sous-structure optimale – Multiplication de matrices

Supposons que la parenthésisation optimale de $A_1 A_2 \cdots A_n$ effectue la dernière multiplication comme $(A_1 \cdots A_k)(A_{k+1} \cdots A_n)$. Alors les parenthésisations de $A_1 \cdots A_k$ et $A_{k+1} \cdots A_n$ dans cette solution optimale sont elles-mêmes optimales.

Proof. Soit $M(P)$ le nombre de multiplications scalaires d'une parenthésisation P. Notons $((O_L) \cdot (O_R))$ la solution optimale où O_L et O_R parenthésisent respectivement $A_1 \cdots A_k$ et $A_{k+1} \cdots A_n$. Soient P_L et P_R des parenthésisations optimales de ces deux sous-chaînes. On a :

$$\begin{aligned} M((O_L) \cdot (O_R)) &= p_0 \cdot p_k \cdot p_n + M(O_L) + M(O_R) \\ &\geq p_0 \cdot p_k \cdot p_n + M(P_L) + M(P_R) = M((P_L) \cdot (P_R)). \end{aligned}$$

Puisque $((O_L) \cdot (O_R))$ est optimal, on a l'égalité, donc $M(P_L) = M(O_L)$ et $M(P_R) = M(O_R)$: P_L et P_R sont bien optimales. □

9.8 Formule récursive

Soit $m[i, j]$ le nombre minimal de multiplications scalaires pour calculer $A_i A_{i+1} \cdots A_j$.

$$m[i,j] = \begin{cases} 0 & \text{si } i = j, \\ \min\limits_{i \le k < j} \{m[i,k] + m[k+1,j] + p_{i-1}\, p_k\, p_j\} & \text{si } i < j. \end{cases}$$

Point Clé 9.2: Ordre de remplissage

$m[i,j]$ ne dépend que de sous-problèmes avec un intervalle $j-i$ *plus petit*. Un algorithme bottom-up résout donc les sous-problèmes par ordre croissant de $j-i$ (longueur de chaîne $\ell = j-i+1$).

9.9 Algorithme bottom-up

Algorithm 8 Matrix-Chain-Order(p)

```
n ← p.length − 1
Créer les tables m[1...n, 1...n] et s[1...n, 1...n]
for i = 1 to n do
    m[i, i] ← 0
end for
for ℓ = 2 to n do                                   // ℓ = longueur de la chaîne
    for i = 1 to n − ℓ + 1 do
        j ← i + ℓ − 1
        m[i, j] ← ∞
        for k = i to j − 1 do
            q ← m[i, k] + m[k + 1, j] + p_{i−1} p_k p_j
            if q < m[i, j] then
                m[i, j] ← q
                s[i, j] ← k                         // s mémorise le choix optimal
            end if
        end for
    end for
end for
return m et s
```

Complexité 9.1: Matrix-Chain-Order

- La table m contient $O(n^2)$ entrées.
- Chaque entrée $m[i,j]$ requiert $O(n)$ pour trouver le meilleur k.
- **Complexité totale : $\Theta(n^3)$ en temps, $\Theta(n^2)$ en espace.**

9.10 Exemple complet

Chaîne de $n = 6$ matrices avec dimensions :

Matrice	A_1	A_2	A_3	A_4	A_5	A_6
Dimensions	30×35	35×15	15×5	5×10	10×20	20×25

Soit $p = \langle 30, 35, 15, 5, 10, 20, 25 \rangle$.

Les tables m (coûts optimaux) et s (choix optimaux) calculées par Matrix-Chain-Order sont :

Table $m[i,j]$

$j \backslash i$	1	2	3	4	5	6
1	0					
2	15 750	0				
3	7 875	2 625	0			
4	9 375	4 375	750	0		
5	11 875	7 125	2 500	1 000	0	
6	15 125	10 500	5 375	3 500	5 000	0

Table $s[i,j]$

$j \backslash i$	1	2	3	4	5	6
2	1					
3	1	2				
4	3	3	3			
5	3	3	3	4		
6	3	3	3	5	5	

La solution optimale est $m[1,6] = 15\,125$ multiplications scalaires. En remontant depuis $s[1,6] = 3$, on obtient la parenthésisation :

$$\big(A_1\,(A_2\,A_3)\big)\big((A_4\,A_5)\,A_6\big)$$

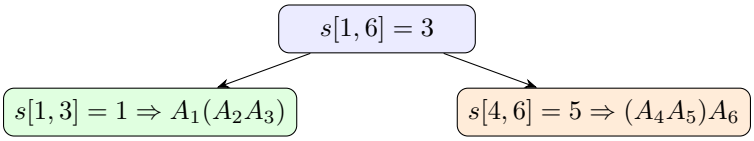
9.11 Reconstruction de la parenthésisation optimale

La table s permet de reconstruire récursivement la parenthésisation :

Algorithm 9 Print-Optimal-Parens(s, i, j)

```
if i = j then
    afficher "A_i"
else
    afficher "("
    PRINT-OPTIMAL-PARENS(s, i, s[i, j])
    PRINT-OPTIMAL-PARENS(s, s[i, j] + 1, j)
    afficher ")"
end if
```

Trace pour l'exemple ($i = 1$, $j = 6$) :



Résultat : $\big(A_1\,(A_2\,A_3)\big)\big((A_4\,A_5)\,A_6\big)$

9.12 Résumé : multiplication de chaînes de matrices

Récapitulatif DP – Multiplication de chaînes de matrices

- **Choix** : où placer la parenthèse la plus externe $(A_1 \cdots A_k)(A_{k+1} \cdots A_n)$.
- **Sous-structure optimale** : les deux sous-chaînes doivent être parenthésées de façon optimale (prouvé par l'argument de cut-and-paste).
- **Récurrence** :

$$m[i,j] = \begin{cases} 0 & \text{si } i = j, \\ \min\limits_{i \le k < j} \{m[i,k] + m[k+1,j] + p_{i-1}\, p_k\, p_j\} & \text{si } i < j. \end{cases}$$

- **Complexité** : $\Theta(n^3)$ en temps, $\Theta(n^2)$ en espace.
- **Reconstruction** : table auxiliaire s + PRINT-OPTIMAL-PARENS.

9.13 Recette générale pour la DP (consolidation)

Problème	Choix	Complexité	Taille de la table
Coupe de tiges	Position de la coupe gauche	$\Theta(n^2)$	$n+1$ entrées
Rendu de monnaie	Dénomination utilisée	$O(nW)$	$W+1$ entrées
Chaînes de matrices	Position de la parenthèse	$\Theta(n^3)$	n^2 entrées

Les trois étapes systématiques restent :

1. Identifier les choix et la sous-structure optimale.
2. Écrire la récurrence pour la valeur optimale.
3. Résoudre efficacement (top-down avec mémoïsation ou bottom-up).